# From Data Discovery to Analytics-Ready Data: A Survey of Search, Preparation, and Integration in Data Lakes

Sumaiya Azad
University of Utah
Salt Lake City, Utah, USA
sumaiya.azad@utah.edu

## ABSTRACT

Data lakes promise flexible storage, but analysts rarely receive data in a form ready for dashboards, statistical analysis, or machine learning. Between raw lake contents and analytics-ready outputs, users must discover relevant sources, prepare noisy and heterogeneous data, and integrate partially overlapping tables. This literature review organizes recent work into three phases: Discovery and Search, Preparation and Wrangling, and Integration and Matching. It summarizes representative methods, explains the basis of the taxonomy, compares trade-offs in accuracy, automation, scalability, and human effort, and highlights open problems in end-to-end evaluation, safe automation, governance, and evolving data. The review shows how current systems move from data-lake search to usable analytical datasets, and where important research gaps remain.

## 1 INTRODUCTION

Data lakes store heterogeneous tables, semi-structured files, logs, spreadsheets, and extracted document fields without forcing a single schema at ingestion time. Existing discovery surveys explain why this flexibility is attractive for business intelligence (BI), scientific analysis, and machine learning (ML), while also making data harder to retrieve, interpret, and combine [2, 14]. Before analysts can answer a question, they must discover relevant sources, prepare them for use, and integrate partial evidence across tables.

This is also a human-centered systems problem. Analysts still interpret weak metadata, validate repairs, inspect suggested joins, and decide whether automated outputs are trustworthy, as reflected across recent discovery and integration systems [24, 28, 32]. Useful systems therefore need more than accuracy; they must reduce manual effort while preserving control when the lake is messy, incomplete, or policy constrained.

Recent database work from 2022 to 2025 has shifted from metadata-driven retrieval and rule-based cleaning toward semantic retrieval, pipeline synthesis, and language-model (LM)-assisted integration [11, 15, 17, 32]. In this survey, I view the literature as a pipeline from discovery to analytics-ready data in lake or closely related lakehouse settings.

Across that pipeline, recurring evaluation criteria include retrieval quality, downstream utility, matching accuracy, scalability, runtime cost, robustness to weak metadata, and the amount of human supervision required. These criteria point to common challenges: large and evolving lakes, ambiguous user intent, unsafe automation, and limited end-to-end evaluation even when individual stages perform well.

*Contributions of this Survey.* The survey organizes recent work as a pipeline from discovery to analytics-ready outputs, groups methods by primary output to form a clear taxonomy, contrasts where major method families work well or fail, and identifies open directions in evaluation, safe automation, governance, and benchmark design.

*Organization.* Section 2 defines the core terminology used throughout the paper. Section 3 surveys prior work and introduces the taxonomy used for categorization. Section 4 compares method families and discusses their trade-offs. Section 5 outlines open challenges and future directions, and Section 6 concludes.

## 2 BACKGROUND AND PRELIMINARIES

### 2.1 Core Terms

A *data lake* is a repository that stores heterogeneous data without requiring a strict schema in advance [2, 14]. *Analytics-ready data* is data that has been cleaned, structured, and integrated well enough to support reporting, statistics, or downstream models. In this review, *Discovery and Search* covers keyword search, join search, union search, and interactive exploration; its outputs are typically ranked tables, columns, views, or candidate augmentations. *Preparation and Wrangling* covers cleaning, transformation, relationalization, and workflow construction; its outputs are cleaned tables, structured views, or rerunnable pipelines. *Integration and Matching* covers schema matching, entity matching (EM), entity resolution (ER), and data integration (DI); its outputs are correspondences, resolved entities, or merged results.

Join search finds tables that can be linked through a shared key, while union search finds tables whose compatible columns allow them to be stacked together [10, 11, 25]. Schema matching decides which columns correspond across tables, ER decides whether two records refer to the same real-world entity, and DI uses those correspondences to produce a combined result [20, 32, 41].

### 2.2 A Pipeline View

The key idea behind this survey is that these tasks form a pipeline. Discovery narrows a large lake to a small set of plausible candidates. Preparation turns those candidates into stable and usable inputs. Integration aligns and merges them into an output suitable for analysis. Treating these stages together matters because errors propagate across them: weak retrieval can surface irrelevant tables, incomplete preparation can preserve format conflicts, and poor integration can collapse useful but partial evidence [24, 28, 35].

## 2.3 Evaluation Criteria

Although each phase uses different metrics, several criteria appear throughout the literature. Discovery studies usually report ranking quality, coverage, and retrieval speed. Preparation studies emphasize net error reduction, downstream task utility, and repeatability of workflows. Integration studies focus on matching quality, merge completeness, supervision cost, fairness, and policy constraints. These criteria motivate the taxonomy and the cross-phase comparison used later in the review.

# 3 SURVEY AND CATEGORIZATION OF PRIOR WORK

## 3.1 Basis of Categorization

This review groups recent work by *primary output*. A method belongs to Discovery and Search if it mainly returns ranked candidates for exploration, to Preparation and Wrangling if it mainly returns cleaned tables or reusable workflows, and to Integration and Matching if it mainly returns correspondences, entity matches, or merged tables. This rule avoids double counting when one system touches more than one stage.

Using that rule, the literature forms three main classes with recurring subcategories. Discovery includes end-to-end search systems, semantic retrieval, join and union search, and benchmark-oriented work. Preparation includes cleaning and repair, pipeline synthesis, and structuring messy inputs. Integration includes schema alignment, ER workflows, fairness-aware evaluation, and reusable integration signals.

## 3.2 Scope of the Reviewed Literature

The reviewed corpus contains 49 papers published between 2022 and 2025 in the SIGMOD International Conference on Management of Data (SIGMOD), the Proceedings of the VLDB Endowment (PVLDB), and the International Conference on Data Engineering (ICDE). Forty-seven are method papers that fit one of the three pipeline phases, and two are survey-style discovery references used to frame terminology and evaluation practice. Papers were selected by keyword groups in Table 1, filtered to lake/lakehouse-relevant tasks, and categorized by the dominant output reported in each paper. Each included method is assigned to exactly one phase by its primary output. This compresses real overlap, but it keeps the taxonomy consistent and makes phase-level comparison easier to interpret.

Table 1 records the four search-term groups used to keep the corpus aligned with the pipeline studied in this review. The first two groups target discovery and relationship search, the third targets preparation and workflow construction, and the fourth targets matching and integration. This table therefore connects corpus construction with the analysis developed in the rest of the survey's categorization and discussion.

With that scope fixed, the rest of this section is structured around three stages. Figure 1 gives the high-level map, Table 2 turns that map into outputs and recurring concerns, and the later tables unpack the representative papers inside each phase.

Figure 1 shows how the literature branches into subproblems, while the Table 2 shows what each branch actually produces for the next stage of the pipeline. Together they explain why the survey moves from finding candidate data, to stabilizing that data, to merging it into an analytics-ready result.

**Table 1: Search term groups used for queries over data lakes.**

| ID | Query family (examples of task keywords) |
|---|---|
| S1 | `discovery`, `search`, `table search`, `dataset retrieval`, `exploration` |
| S2 | `joinable`, `unionable`, `overlap`, `join discovery`, `union discovery` |
| S3 | `wrangling`, `preparation`, `cleaning`, `transform`, `pipeline synthesis` |
| S4 | `schema matching`, `ER`, `record linkage`, `DI`, `LLM` |

## 3.3 Discovery and Search

Discovery is the opening stage of the workflow: before any repair or integration can happen, the analyst needs a plausible set of candidate tables, columns, or views. Table 3 organizes this stage by showing how the literature moves from end-to-end search systems, to learned retrieval signals, to join/union relationship search, and finally to the shared resources that make discovery evaluation possible. The categories show the main lines of work, while the representative papers illustrate the retrieval problems each line emphasizes.

Discovery research addresses the first bottleneck in data lakes: users may know the question they want to answer but still not know which tables are relevant, joinable, or worth inspecting [2, 14]. In practice, analysts usually begin with an information need such as "find customer-address data" or "find tables that can enrich sales records," not with a precise table name. Discovery methods therefore combine three jobs: representing what a table contains, retrieving plausible candidates, and helping the user refine or trust the returned results.

The first category in Table 3 captures end-to-end systems such as LakeCompass, BLEND, CHORUS, METAM, Selecting Subtables, and Ver. Across these papers, discovery is treated as more than ranking: systems maintain indexes, generate previews, support interaction, or optimize for downstream utility so that the analyst can decide whether a candidate should move deeper into the pipeline.

A second category replaces surface similarity with richer semantics. CMDL, Starmie, LIFTus, and TabSketchFM learn representations that remain useful when labels are sparse, abbreviated, or inconsistent. DeepJoin, SANTOS, Nested/Joined Lake Search, and Armadillo extend that shift to relationship search by ranking tables for joinability, unionability, or overlap rather than surface resemblance alone. These methods are most helpful when lexical similarity is weak but the returned tables still need to support a later join, augmentation, or merge.

Discovery also depends on shared infrastructure. Rottnest, GitTables, and LakeBench support indexing, large-scale evaluation, and reproducible workloads, while the recent discovery surveys help stabilize terminology and task definitions. Strong ranking alone is not enough: discovery is only successful when the returned candidates reduce downstream preparation work instead of creating more of it.

## 3.4 Preparation and Wrangling

Once discovery has narrowed the lake, the survey moves to the middle stage: turning promising candidates into dependable inputs.

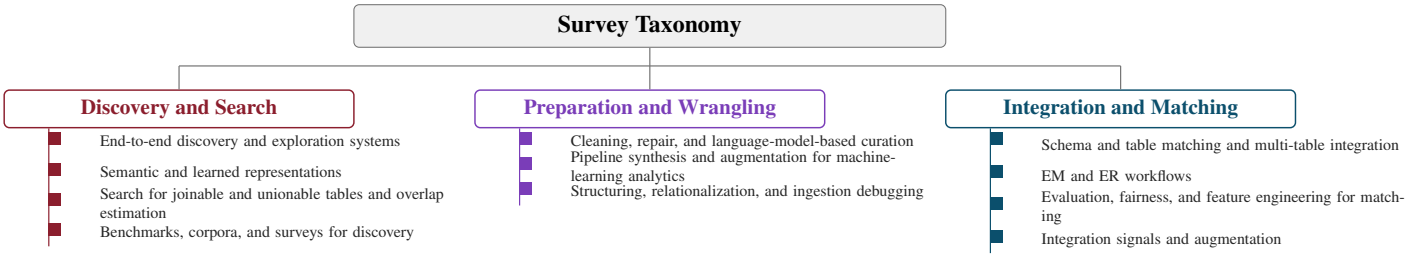



**Figure 1: Taxonomy of data lake research from discovery and search to analytics-ready outputs via preparation and integration.**

**Table 2: Phase-level summary of outputs, recurring tasks, and common concerns.**

| Phase | Primary output | Recurring tasks | Common concerns |
|---|---|---|---|
| Discovery and Search | Ranked candidate tables, columns, views, or augmentations | Keyword search, join search, union search, overlap estimation, view discovery, interactive exploration | Weak metadata, large search space, ambiguity, index maintenance, and runtime cost |
| Preparation and Wrangling | Cleaned tables, structured views, transformed tables, or rerunnable workflows | Repair, transformation recommendation, pipeline synthesis, feature augmentation, relationalization, ingestion debugging | Unsafe edits, limited validation, repeatability, downstream utility, and evolving data |
| Integration and Matching | Column matches, entity matches, merge plans, reusable evidence, or merged outputs | Schema matching, ER, blocking, merge-rule design, external augmentation, fairness auditing | Supervision cost, partial overlap, fairness, privacy, policy constraints, and intent sensitivity |

**Table 3: Discovery & Search papers mapped to taxonomy to show category structure and representative contributions.**

| Category | Reference | Keywords | Contribution |
|---|---|---|---|
| End-to-end discovery and exploration systems | CHORUS [24] | LMs, annotation | Unifies multiple lake discovery/exploration tasks with a foundation-model pipeline and reports consistent gains over task-specific baselines. |
| | BLEND [13] | plans, ranking | Defines a declarative discovery-plan abstraction that compiles to improved execution over heterogeneous signals for unified keyword, join, and union discovery with top-$k$ retrieval. |
| | LakeCompass [7] | indexing, workflows | End-to-end lake stack for maintenance, search, and analysis workflows, with scalable indexing and practical lake operations. |
| | METAM [16] | utility, ranking | Ranks candidate datasets by expected downstream task utility through goal-aware profiling and ranking. |
| | Selecting Sub-tables [3] | sampling, previews | Selects compact, informative sub-tables to accelerate inspection when full tables are too large or noisy. |
| | Ver [21] | views, interaction | Question-driven view discovery that progressively prunes the space to surface useful derived tables. |
| Semantic and learned representations | CMDL [12] | cross-modal, embeddings | Learns joint representations to retrieve across structured tables and unstructured text sources for cross-modal dataset discovery. |
| | Starmie [15] | embeddings, contrastive | Pretrains contextualized column representations that capture table context to improve semantic dataset discovery, especially unionable-table retrieval. |
| | LIFTus [38] | fusion, columns | Learns multi-aspect column representations and adaptively fuses them to boost union search quality across heterogeneous lakes. |
| | TabSketchFM [26] | sketches, tokens | Uses compact column sketches with a pretrained tabular model to scale discovery under compute constraints. |
| Search for joinable and unionable tables and overlap estimation | DeepJoin [11] | joinability, retrieval | Pre-trained-language-model-based joinable-table discovery that embeds columns to rank candidate join partners, improving join discovery over lexical baselines. |
| | SANTOS [25] | unionability, semantics | Relationship-aware semantic union search using relational/context signals to find unionable tables with compatible meaning. |
| | Nested/Joined Lake Search [48] | nested, joins | Extends lake search to nested and join-derived candidates not well-modeled as flat standalone tables. |
| | Armadillo [37] | overlap, graphs | Learns table embeddings whose similarity approximates overlap ratio for fast scalable overlap estimation. |
| Benchmarks, corpora, and surveys for discovery | Rottnest [43] | indexing, search | Scalable lake indexing techniques to support efficient search over large, evolving lakes. |
| | GitTables [22] | corpus, benchmark | Large public corpus of relational tables mined at scale for reproducible training and evaluation. |
| | LakeBench [10] | benchmark, labels | Benchmark suite for joinable and unionable table discovery with ground-truth labeling at lake scale. |
| | Data Discovery Survey [2] | survey, taxonomy | Consolidates discovery operations across indexes and systems, and reviews evaluation practice while noting gaps in workloads and cost reporting. |
| | Table Discovery Survey [14] | survey, methods | Consolidated overview of table discovery methods and open directions at lake scale. |

Table 4 connects papers that might otherwise look separate by showing that repair, workflow synthesis, relationalization, and ingestion debugging all answer the same question: how do we make selected data usable enough for later analysis and integration? Its categories mark the main preparation tasks, and the representative papers show the kinds of usable outputs each task is designed to produce.

Preparation and Wrangling changes the data itself. Where discovery returns a ranked list of candidates, preparation produces cleaned

**Table 4: Preparation & Wrangling methods mapped to taxonomy to show category structure and representative contributions.**

| Category | Reference | Keywords | Contribution |
|---|---|---|---|
| Cleaning, repair, and language-model-based curation | Automatic Data Repair [35] | repair, benchmark | Benchmarks repair algorithms across datasets/error settings, proposes a repair-search strategy, and derives deployment guidelines. |
| | DataRinse [1] | transforms, mining | Recommends semantic data-prep transformations by mining data-preparation code repositories to learn reusable transformation patterns. |
| | Foundation Models for Data Tasks [34] | LMs, prompting | Tests prompting-based use of foundation models on several cleaning and integration tasks, while surfacing limits on private and domain-specific data. |
| | Lingua Manga [9] | LLMs, templates | Provides an LM-based curation system with modular templates and orchestration to run end-to-end curation workflows. |
| Pipeline synthesis and augmentation for ML analytics | CtxPipe [17] | pipelines, reinforcement learning (RL) | Constructs preparation pipelines by conditioning operator selection on dataset/task context and using deep RL to search effective pipelines. |
| | HAIPipe [8] | human-machine collaboration, reuse | Combines human- and machine-generated pipelines via enumeration–sampling (with active learning) and uses RL to search improved machine-generated pipelines. |
| | Featpilot [30] | features, joins | Automatically augments tabular datasets with useful additional features from joinable sources to improve downstream model performance. |
| | Automatically Build Business-Intelligence Models (Auto-BI) [31] | BI, joins | Predicts BI schema graphs (joins) via global search, evaluated on large collections of BI models. |
| Structuring, relationalization, and ingestion debugging | Evaporate [4] | views, synthesis | Uses LMs plus lightweight synthesis and weak supervision to generate structured views over heterogeneous lake inputs. |
| | Auto-Tables [29] | relationalization, schema | Relationalizes semi-structured tables without examples by inferring structure/schemas and producing normalized relational output. |
| | ReCG [47] | JavaScript Object Notation (JSON), clustering | Discovers JSON schemas bottom-up by clustering records and generalizing clusters into schema fragments. |
| | CM Explorer [6] | validation, debugging | Disentangles correlated violated conditional unit tests using graph-based exploration to localize ingestion errors. |
| | Fast and Scalable Data Transfer Across Data Systems [18] | transfer, tuning | Decomposes data transfer into configurable stages and auto-tunes configurations for heterogeneous systems to improve throughput. |

tables, derived features, structured views, or rerunnable workflows that can support repeated analysis. This stage matters because many errors that look small in isolation, such as inconsistent date formats or semi-structured headers, become serious blockers once matching or integration begins [1, 29, 35].

One major strand in Table 4 emphasizes repair and transformation reuse. Automatic Data Repair focuses on safer repairs, DataRinse mines reusable transforms, foundation-model prompting broadens coverage across cleaning tasks, and Lingua Manga orchestrates end-to-end curation workflows. Across all four, the difficult step is not generating candidate edits but checking whether those edits are correct, explainable, and repeatable.

Another strand treats preparation as workflow construction. CtxPipe, HAIPipe, Featpilot, and Auto-BI search for pipelines, join paths, or augmentations that improve recurring analytical tasks. Evaporate, Auto-Tables, ReCG, CM Explorer, and Fast and Scalable Data Transfer Across Data Systems instead focus on extracting structure from messy inputs, debugging ingestion failures, or tuning movement across systems. In these papers, preparation quality is measured less by local cleanliness than by whether the resulting tables are easier to analyze, join, or reuse.

Preparation therefore exposes a tension between quick automation and reusable process design. LM-based curation can shorten time to first result, but repeated use still requires provenance, validation, and rerunnable workflows. Pipeline-construction systems help when the same analytical pattern recurs across datasets, whereas relationalization systems matter when later matching depends on structure that is still implicit in the raw inputs.

## 3.5 Integration and Matching

Integration is the stage where cleaned candidates must become one coherent analytical output. Table 5 groups this stage into schema/table integration, ER workflows, evaluation-oriented matching work, and reusable integration signals. The categories separate the main integration choices, and the representative papers show which matching or merge issues those choices bring to the foreground.

What makes this stage difficult is not candidate retrieval but alignment under weak and partial evidence. In data lakes, metadata is often sparse, tables overlap only partially, and two analysts may want different definitions of what counts as the "same" entity or the "same" schema [20, 32, 39]. Table 5 therefore groups the literature by the kind of evidence each system uses before merging fragments.

DIALITE and ALITE keep merge semantics in view after discovery, while Magneto and the PLM schema-matching method strengthen alignment with LM or embedding signals. In all four cases, a column match is only an intermediate result: the system still has to decide how partial tuples, open-world values, and schema differences are carried into the merged output.

FlexER, Progressive EM, Unicorn, WeakSup GT Inference, and Battleship treat matching as workflow design under varying intents, budgets, and supervision levels. The surrounding papers in this section add fairness audits, blocking benchmarks, feature rules, enrichment methods, external augmentation, self-supervised representations, causal augmentation, and similarity constraints. The field now studies not only how to classify a pair, but also which signals should be collected and trusted before a merge is accepted.

Correctness is also highly task dependent at this stage. A merged result can be technically well aligned but still analytically wrong if it collapses partial evidence, applies the wrong entity granularity, or

**Table 5: Integration & Matching methods mapped to taxonomy to show category structure and representative contributions.**

| Category | Reference | Keywords | Contribution |
|---|---|---|---|
| Schema and table matching and multi-table integration | DIALITE [28] | open-data, alignment | End-to-end pipeline that discovers relevant open tables and integrates them by aligning schemas and values with knowledge-graph and LLM assistance. |
| | ALITE [27] | integration, disjunction | Adaptive-learning approach for integrating discovered lake tables under full-disjunction semantics at scale. |
| | Magneto [32] | matching, hybrid LMs | Stages schema matching in two steps: a smaller LM proposes candidate matches, and a larger LM reviews ambiguous cases. |
| | Pre-trained Language Model (PLM) Schema Matching [49] | matching, embeddings | Adapts PLMs to encode attribute and column semantics for schema matching across heterogeneous schemas. |
| EM and ER workflows | Flexible Entity Resolution (FlexER) [20] | ER, intents | Models ER with multiple downstream intents as multi-label classification and uses a multiplex graph neural network (GNN) for intent-aware matching. |
| | Progressive EM [33] | progressive, scheduling | Characterizes the design space of progressive EM and benchmarks progressive strategies including neural-network- and PLM-based variants. |
| | Unicorn [41] | multitask, matching | Unified multi-task model supporting several DI matching tasks via shared encoders and task-specific heads. |
| | WeakSup GT Inference [44] | weak-supervision, labels | Infers latent ground truth from weak supervision sources by learning a label model that combines noisy signals for EM. |
| | Battleship [19] | active-learning, low-resource | Active-learning approach for low-resource EM that selects informative pairs via a Battleship-style strategy. |
| Evaluation, fairness, and feature engineering for matching | Fairness Lens [39] | fairness, bias | Builds social datasets for EM and uses them to measure how error rates differ across groups under class imbalance. |
| | Filtering Benchmarks [36] | blocking, benchmark | Benchmarks combinations of filtering and scheduling strategies for progressive ER across datasets. |
| | DescSim Rules [40] | rules, features | Introduces description-similarity rules to engineer informative, interpretable EM features from attribute descriptions. |
| | Relation Enrichment [45] | enrichment, attributes | Boosts ER quality by enriching related tuples with additional attributes from additional sources. |
| Integration signals and augmentation | Causal Data Integration (CDI) [46] | causal, confounders | Defines CDI by augmenting datasets with missing confounders from external sources and inferring a causal graph. |
| | Entity Augmentation [5] | augmentation, querying | Studies how to augment entities by selectively querying external sources with cost-aware querying strategies. |
| | Sudowoodo [42] | contrastive, representations | Self-supervised contrastive approach that learns similarity-aware representations from unlabeled data and transfers to multiple DI tasks. |
| | Similarity Inclusion Dependencies [23] | dependencies, profiling | Discovers similarity inclusion dependencies, which act as approximate containment or foreign-key-like signals to reveal join and integration opportunities. |

hides ambiguity that a user would want to inspect. Recent matching work therefore spends more attention on intent, fairness, supervision cost, and reusable evidence, not only on final accuracy.

## 4 COMPARISON AND DISCUSSION

By this point, the earlier tables have introduced the major method families phase by phase. Table 6 brings them back together and asks the higher-level survey question: under what scenarios does each family help most, and where do its strengths stop being enough?

Table 6 shows that the major method families solve different bottlenecks and should be compared by scenario rather than by one universal metric. Discovery families help when the lake is large and weakly documented, preparation families matter when cleaning logic or workflow structure must be reused, and integration families matter when partial overlap, ambiguous matches, or policy constraints dominate.

This comparison also clarifies why older design choices remain relevant. Metadata-driven discovery is still attractive when lakes are stable and descriptive, rule-based repair remains useful when strong domain constraints are available, and explicit merge semantics remain valuable when partial facts must be preserved. Learned methods become more attractive when labels are sparse, schemas vary, or semantics drift across sources, but they usually demand more validation, more compute, and more user trust [15, 32, 35].

Within discovery, the main contrast is between operational breadth and semantic depth. Systems centered on indexes, plans, and previews are often easier to monitor and maintain, and they work well when table names, keys, or metadata remain reasonably informative over time. Learned retrieval becomes more valuable when those signals are weak or inconsistent, because column content and context carry more meaning than names alone. The trade-off is that explanations become harder. A user may receive a strong candidate without understanding which structural or semantic cues made it appear reliable, which can slow trust and downstream adoption [7, 15].

Preparation methods divide more sharply by intended use. Repair and transformation systems are most effective when analysts repeatedly confront noisy values, format inconsistencies, or missing fields. Pipeline-synthesis systems are better suited to repeated ML or BI tasks where reusable operator sequences matter as much as the final table. Structuring and relationalization systems solve a different problem again: they create the tabular form that later discovery and matching methods assume already exists. In practice, one family is not "better" than another unless the analyst's bottleneck is clear, because each family removes a different source of difficulty [17, 29, 35].

Integration methods also differ in where their difficulty lies. Some problems are dominated by schema heterogeneity and benefit from stronger column representations or staged matching. Others are dominated by limited labels, so weak supervision, active learning, or reusable similarity signals become more useful than a slightly stronger final classifier. In settings with partial overlap, open data, or enterprise policy constraints, explicit merge semantics may matter more than raw matching accuracy. A system that exposes how partial

**Table 6: Cross-phase comparison of representative method families.**

| Family | Representative works | Works well when | Main strengths | Main limitations |
|---|---|---|---|---|
| End-to-end discovery systems | LakeCompass [7], BLEND [13], CHORUS [24] | Users need broad search over large lakes | Unified search and exploration stack | Maintenance cost and intent drift |
| Learned discovery and relationship search | Starmie [15], DeepJoin [11], SANTOS [25] | Semantic overlap matters more than exact names | Robust to weak metadata | Training cost and lower interpretability |
| Cleaning and workflow automation | Automatic Data Repair [35], CtxPipe [17], Lingua Manga [9] | Preparation tasks repeat across datasets | Reusable workflows and less manual work | Hard validation and unsafe edits |
| Structuring and relationalization | Evaporate [4], Auto-Tables [29], ReCG [47] | Inputs are semi-structured or irregular | Makes messy data queryable | Brittle multi-step transforms |
| Integration and ER workflows | DIALITE [28], Magneto [32], FlexER [20], Unicorn [41] | Sources overlap under limited labels or policy constraints | Richer outputs and intent awareness | Merge semantics, fairness, and privacy |

facts are preserved can be more valuable than one that reports a higher score but hides important modeling assumptions [19, 28, 44].

Across all three phases, the key question is whether a method leaves the next stage easier rather than harder.

# 5 OPEN CHALLENGES AND FUTURE DIRECTIONS

*End-to-End Evaluation.* Most papers still report metrics for a single phase. A major need is benchmark design that records stage-level metrics together with at least one end-to-end analytical outcome; without that broader view, phase-specific scores can make methods look more comparable than they really are.

*Intent, Interaction, and Safe Automation.* Analytical intent is often ambiguous in data lakes, so future systems should support interactive refinement and make uncertainty visible instead of hiding it [20, 24]. This matters even more as language-model-based discovery, preparation, and integration become common [9, 32]. The open problem is not only how to automate more, but how to automate safely through validation hooks, rollback mechanisms, constraint checks, and confidence-aware execution. Analysts should be able to inspect competing joins, repairs, and match definitions before those choices become part of a final dataset.

*Scale, Maintenance, and Governance.* At realistic lake scale, infrastructure costs matter as much as model quality. Discovery systems must maintain indexes and embeddings, preparation systems may execute many candidate workflows, and integration systems often spend most of their time in blocking, filtering, or label acquisition [7, 17, 36]. Future work should report speed-quality trade-offs explicitly and incorporate governance, fairness, and privacy requirements into the workflow design rather than checking them only after results are produced [39]. Incremental maintenance is especially important when lakes evolve continuously instead of by occasional rebuilds.

*Evolving Lakes and Shared Resources.* Many systems still assume that the analytical challenge begins only after data has already been made tabular, but real lakes evolve in format, nesting depth, and storage location over time [4, 48]. Future work should connect discovery, preparation, and integration more directly for semi-structured and changing data. It also needs shared test collections that cover the full pipeline, maintenance cost, and boundary failures under evolving-lake conditions.

# 6 CONCLUSION

Recent data-lake research now supports a clear three-phase view: discovery narrows the search space, preparation stabilizes candidate data, and integration aligns and merges the final sources. The main lesson is practical rather than taxonomic: systems should be judged by how reliably they turn retrieved candidates into usable analytical datasets, especially when automation, intent, and governance interact across phase boundaries.

# REFERENCES


[1] Ibrahim Abdelaziz, Julian Dolby, Udayan Khurana, Horst Samulowitz, and Kavitha Srinivas. 2023. DataRinse: Semantic Transforms for Data Preparation Based on Code Mining. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 4090âĂŞ4093. https://doi.org/10.14778/3611540.3611628

[2] Ziawasch Abedjan, Mahdi Esmailoghli, and Sainyam Galhotra. 2025. Data Discovery in Data Lakes: Operations, Indexes, Systems. *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 5455âĂŞ5459. https://doi.org/10.14778/3750601.3750694

[3] Yael Amsterdamer, Susan B. Davidson, Tova Milo, Kathy Razmadze, and Amit Somech. 2023. Selecting Sub-tables for Data Exploration. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2496–2509. https://doi.org/10.1109/ICDE55515.2023.00192

[4] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. 2023. Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 92âĂŞ105. https://doi.org/10.14778/3626292.3626294

[5] Christopher Buss, Jasmin Mousavi, Mikhail Tokarev, Arash Termehchy, David Maier, and Stefan Lee. 2023. Effective Entity Augmentation by Querying External Data Sources. *Proc. VLDB Endow.* 16, 11 (July 2023), 3404âĂŞ3417. https://doi.org/10.14778/3611479.3611535

[6] Niels Bylois, Frank Neven, and Stijn Vansummeren. 2023. CM-Explorer: Dissecting Data Ingestion Problems. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 3958âĂŞ3961. https://doi.org/10.14778/3611540.3611595

[7] Chengliang Chai, Yuhao Deng, Yutong Zhan, Ziqi Cao, Yuanfang Zhang, Lei Cao, Yuping Wang, Zhiwei Zhang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeCompass: An End-to-End System for Data Maintenance, Search and Analysis in Data Lakes. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 4381âĂŞ4384. https://doi.org/10.14778/3685800.3685880

[8] Sibei Chen, Nan Tang, Ju Fan, Xuemi Yan, Chengliang Chai, Guoliang Li, and Xiaoyong Du. 2023. HAIPipe: Combining Human-generated and Machine-generated Pipelines for Data Preparation. *Proc. ACM Manag. Data* 1, 1, Article 91 (May 2023), 26 pages. https://doi.org/10.1145/3588945

[9] Zui Chen, Lei Cao, and Sam Madden. 2023. Lingua Manga: A Generic Large Language Model Centric System for Data Curation. *Proc. VLDB Endow.* 16, 12 (Aug. 2023), 4074âĂŞ4077. https://doi.org/10.14778/3611540.3611624

[10] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *Proc. VLDB Endow.* 17, 8 (April 2024), 1925âĂŞ1938. https://doi.org/10.14778/3659437.3659448

[11] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2023. DeepJoin: Joinable Table Discovery with Pre-Trained Language Models. *Proc. VLDB Endow.* 16, 10 (June 2023), 2458âĂŞ2470. https://doi.org/10.14778/3603581.3603587

[12] Mohamed Y. Eltabakh, Mayuresh Kunjir, Ahmed K. Elmagarmid, and Mohammad Shahmeer Ahmad. 2023. Cross Modal Data Discovery over Structured and Unstructured Data Lakes. *Proc. VLDB Endow.* 16, 11 (July 2023), 3377âĂŞ3390. https://doi.org/10.14778/3611479.3611533

[13] Mahdi Esmailoghli, Christoph Schnell, RenÃľe J. Miller, and Ziawasch Abedjan. 2025. BLEND: A Unified Data Discovery System. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 737–750. https://doi.org/10.1109/ICDE65448.2025.00061

[14] Grace Fan, Jin Wang, Yuliang Li, and Renée J. Miller. 2023. Table Discovery in Data Lakes: State-of-the-art and Future Directions. In *Companion of the 2023 International Conference on Management of Data (SIGMOD '23)*. Association for Computing Machinery, New York, NY, USA, 69âĂŞ75. https://doi.org/10.1145/3555041.3589409

[15] Grace Fan, Jin Wang, Yuliang Li, Dan Zhang, and Renée J. Miller. 2023. Semantics-Aware Dataset Discovery from Data Lakes with Contextualized Column-Based Representation Learning. *Proc. VLDB Endow.* 16, 7 (March 2023), 1726âĂŞ1739. https://doi.org/10.14778/3587136.3587146

[16] Sainyam Galhotra, Yue Gong, and Raul Castro Fernandez. 2023. Metam: Goal-Oriented Data Discovery. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 2780–2793. https://doi.org/10.1109/ICDE55515.2023.00213

[17] Haotian Gao, Shaofeng Cai, Tien Tuan Anh Dinh, Zhiyong Huang, and Beng Chin Ooi. 2024. CtxPipe: Context-aware Data Preparation Pipeline Construction for Machine Learning. *Proc. ACM Manag. Data* 2, 6, Article 231 (Dec. 2024), 27 pages. https://doi.org/10.1145/3698831

[18] Haralampos Gavriilidis, Kaustubh Beedkar, Matthias Boehm, and Volker Markl. 2025. Fast and Scalable Data Transfer Across Data Systems. *Proc. ACM Manag. Data* 3, 3, Article 157 (June 2025), 28 pages. https://doi.org/10.1145/3725294

[19] Bar Genossar, Avigdor Gal, and Roee Shraga. 2023. The Battleship Approach to the Low Resource Entity Matching Problem. *Proc. ACM Manag. Data* 1, 4, Article 224 (Dec. 2023), 25 pages. https://doi.org/10.1145/3626711

[20] Bar Genossar, Roee Shraga, and Avigdor Gal. 2023. FlexER: Flexible Entity Resolution for Multiple Intents. *Proc. ACM Manag. Data* 1, 1, Article 42 (May 2023), 27 pages. https://doi.org/10.1145/3588722

[21] Yue Gong, Zhiru Zhu, Sainyam Galhotra, and Raul Castro Fernandez. 2023. Ver: View Discovery in the Wild. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 503–516. https://doi.org/10.1109/ICDE55515.2023.00045

[22] Madelon Hulsebos, Çagatay Demiralp, and Paul Groth. 2023. GitTables: A Large-Scale Corpus of Relational Tables. *Proc. ACM Manag. Data* 1, 1, Article 30 (May 2023), 17 pages. https://doi.org/10.1145/3588710

[23] Youri Kaminsky, Eduardo H. M. Pena, and Felix Naumann. 2023. Discovering Similarity Inclusion Dependencies. *Proc. ACM Manag. Data* 1, 1, Article 75 (May 2023), 24 pages. https://doi.org/10.1145/3588929

[24] Moe Kayali, Anton Lykov, Ilias Fountalis, Nikolaos Vasiloglou, Dan Olteanu, and Dan Suciu. 2024. Chorus: Foundation Models for Unified Data Discovery and Exploration. *Proc. VLDB Endow.* 17, 8 (April 2024), 2104âĂŞ2114. https://doi.org/10.14778/3659437.3659461

[25] Aamod Khatiwada, Grace Fan, Roee Shraga, Zixuan Chen, Wolfgang Gatterbauer, Renée J. Miller, and Mirek Riedewald. 2023. SANTOS: Relationship-based Semantic Table Union Search. *Proc. ACM Manag. Data* 1, 1, Article 9 (May 2023), 25 pages. https://doi.org/10.1145/3588689

[26] Aamod Khatiwada, Harsha Kokel, Ibrahim Abdelaziz, Subhajit Chaudhury, Julian Dolby, Oktie Hassanzadeh, Zhenhan Huang, Tejaswini Pedapati, Horst Samulowitz, and Kavitha Srinivas. 2025. TabSketchFM: Sketch-Based Tabular Representation Learning for Data Discovery Over Data Lakes. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1523–1536. https://doi.org/10.1109/ICDE65448.2025.00118

[27] Aamod Khatiwada, Roee Shraga, Wolfgang Gatterbauer, and Renée J. Miller. 2022. Integrating Data Lake Tables. *Proc. VLDB Endow.* 16, 4 (Dec. 2022), 932âĂŞ945. https://doi.org/10.14778/3574245.3574274

[28] Aamod Khatiwada, Roee Shraga, and Renée J. Miller. 2023. DIALITE: Discover, Align and Integrate Open Data Tables. In *Companion of the 2023 International Conference on Management of Data (SIGMOD '23)*. Association for Computing Machinery, New York, NY, USA, 187âĂŞ190. https://doi.org/10.1145/3555041.3589732

[29] Peng Li, Yeye He, Cong Yan, Yue Wang, and Surajit Chaudhuri. 2024. Auto-Tables: Relationalize Tables without Using Examples. *SIGMOD Rec.* 53, 1 (May 2024), 76âĂŞ85. https://doi.org/10.1145/3665252.3665269

[30] Jiaming Liang, Chuan Lei, Xiao Qin, Jiani Zhang, Asterios Katsifodimos, Christos Faloutsos, and Huzefa Rangwala. 2025. Featpilot: Automatic Feature Augmentation on Tabular Data. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 2148–2160. https://doi.org/10.1109/ICDE65448.2025.00163

[31] Yiming Lin, Yeye He, and Surajit Chaudhuri. 2023. Auto-BI: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph. *Proc. VLDB Endow.* 16, 10 (June 2023), 2578âĂŞ2590. https://doi.org/10.14778/3603581.3603596

[32] Yurong Liu, Eduardo H. M. Pena, Aécio Santos, Eden Wu, and Juliana Freire. 2025. Magneto: Combining Small and Large Language Models for Schema Matching. *Proc. VLDB Endow.* 18, 8 (April 2025), 2681âĂŞ2694. https://doi.org/10.14778/3742728.3742757

[33] Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, and Yannis Velegrakis. 2025. Progressive Entity Matching: A Design Space Exploration. *Proc. ACM Manag. Data* 3, 1, Article 65 (Feb. 2025), 25 pages. https://doi.org/10.1145/3709715

[34] Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *Proc. VLDB Endow.* 16, 4 (Dec. 2022), 738âĂŞ746. https://doi.org/10.14778/3574245.3574258

[35] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *Proc. VLDB Endow.* 17, 10 (June 2024), 2617âĂŞ2630. https://doi.org/10.14778/3675034.3675051

[36] George Papadakis, Marco Fisichella, Franziska Schoger, George Mandilaras, Nikolaus Augsten, and Wolfgang Nejdl. 2023. Benchmarking Filtering Techniques for Entity Resolution. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 653–666. https://doi.org/10.1109/ICDE55515.2023.00389

[37] Francesco Pugnaloni, Luca Zecchini, Matteo Paganelli, Matteo Lissandrini, Felix Naumann, and Giovanni Simonini. 2025. Table Overlap Estimation through Graph Embeddings. *Proc. ACM Manag. Data* 3, 3, Article 228 (June 2025), 25 pages. https://doi.org/10.1145/3725365

[38] Ermu Qiu, Jun Gao, Yaofeng Tu, and Jingru Yang. 2025. LIFTus: An Adaptive Multi-Aspect Column Representation Learning for Table Union Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 2174–2187. https://doi.org/10.1109/ICDE65448.2025.00165

[39] Nima Shahbazi, Nikola Danevski, Fatemeh Nargesian, Abolfazl Asudeh, and Divesh Srivastava. 2023. Through the Fairness Lens: Experimental Analysis and Evaluation of Entity Matching. *Proc. VLDB Endow.* 16, 11 (July 2023), 3279âĂŞ3292. https://doi.org/10.14778/3611479.3611525

[40] Yafeng Tang, Zheng Liang, Hongzhi Wang, Xiaoou Ding, Tianyu Mu, and Huan Hu. 2025. Description-Similarity Rules: Towards Flexible Feature Engineering for Entity Matching. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1–15. https://doi.org/10.1109/ICDE65448.2025.00008

[41] Jianhong Tu, Ju Fan, Nan Tang, Peng Wang, Guoliang Li, Xiaoyong Du, Xiaofeng Jia, and Song Gao. 2023. Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration. *Proc. ACM Manag. Data* 1, 1, Article 84 (May 2023), 26 pages. https://doi.org/10.1145/3588938

[42] Runhui Wang, Yuliang Li, and Jin Wang. 2023. Sudowoodo: Contrastive Self-supervised Learning for Multi-purpose Data Integration and Preparation. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 1502–1515. https://doi.org/10.1109/ICDE55515.2023.00391

[43] Ziheng Wang, Sasha Krassovsky, Conor Kennedy, Alex Aiken, Weston Pace, Rain Jiang, Huayi Zhang, Chenyu Jiang, and Wei Xu. 2025. Rottnest: Indexing Data Lakes for Search. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 1814–1827. https://doi.org/10.1109/ICDE65448.2025.00139

[44] Renzhi Wu, Alexander Bendeck, Xu Chu, and Yeye He. 2023. Ground Truth Inference for Weakly Supervised Entity Matching. *Proc. ACM Manag. Data* 1, 1, Article 32 (May 2023), 28 pages. https://doi.org/10.1145/3588712

[45] Mengyi Yan, Wenfei Fan, Yaoshu Wang, and Min Xie. 2024. Enriching Relations with Additional Attributes for ER. *Proc. VLDB Endow.* 17, 11 (July 2024), 3109âĂŞ3123. https://doi.org/10.14778/3681954.3681987

[46] Brit Youngmann, Michael Cafarella, Babak Salimi, and Anna Zeng. 2023. Causal Data Integration. *Proc. VLDB Endow.* 16, 10 (June 2023), 2659âĂŞ2665. https://doi.org/10.14778/3603581.3603602

[47] Joohyung Yun, Byungchul Tak, and Wook-Shin Han. 2024. ReCG: Bottom-up JSON Schema Discovery Using a Repetitive Cluster-and-Generalize Framework. *Proc. VLDB Endow.* 17, 11 (July 2024), 3538âĂŞ3550. https://doi.org/10.14778/3681954.3682019

[48] Yi Zhang, Peter Baile Chen, and Zachary G. Ives. 2024. Searching Data Lakes for Nested and Joined Data. *Proc. VLDB Endow.* 17, 11 (July 2024), 3346âĂŞ3359. https://doi.org/10.14778/3681954.3682005

[49] Yunjia Zhang, Avrilia Floratou, Joyce Cahoon, Subru Krishnan, Andreas C. MÃijller, Dalitso Banda, Fotis Psallidas, and Jignesh M. Patel. 2023. Schema Matching using Pre-Trained Language Models. In *2023 IEEE 39th International Conference on Data Engineering (ICDE)*. 1558–1571. https://doi.org/10.1109/ICDE55515.2023.00123